Atelier 7 : Pipeline CI/CD avec GitLab et SonarQube Cloud

Rapport de projet · [Votre nom] · Novembre 2026

1. Contexte et objectifs

Ce projet a pour but de mettre en place un pipeline d'intégration et de déploiement continus (CI/CD) sur GitLab, avec un contrôle automatique de la qualité et de la sécurité du code grâce à SonarQube Cloud. Le principe : un code conforme est déployé automatiquement sur GitLab Pages, alors qu'un code vulnérable est bloqué avant le déploiement.

Pour l'illustrer, j'ai réalisé un mini-site web en deux versions :

- **V1** : un code correct, analysé puis déployé.
- **V2** : un code volontairement vulnérable (faille XSS), dont le déploiement doit être bloqué.

2. Technologies utilisées

- HTML, CSS, JavaScript pour le mini-site
- Git et GitLab pour le dépôt de code
- GitLab CI/CD (fichier `.gitlab-ci.yml`) pour le pipeline
- SonarQube Cloud pour l'analyse statique et le Quality Gate
- GitLab Pages pour l'hébergement du site

3. Architecture du pipeline

Le flux de travail se déroule ainsi :

1. Je pousse mon code avec `git push`.
2. GitLab déclenche automatiquement le pipeline.
3. Le scanner SonarQube analyse le code et envoie les résultats à SonarQube Cloud.
4. SonarQube évalue le Quality Gate (avec `sonar.qualitygate.wait=true`).
5. Si le Quality Gate est réussi, le job de déploiement publie le site. Sinon, le pipeline s'arrête.

La V1 utilise deux jobs : `sonarqube-check` (stage `quality`) et `deploy-pages` (stage `deploy`). La V2 ajoute un stage `security` et deux jobs : `syntax-check` (vérification de la syntaxe avec `node --check`) et `security-check` (interrogation de l'API SonarQube pour compter les vulnérabilités et les hotspots).

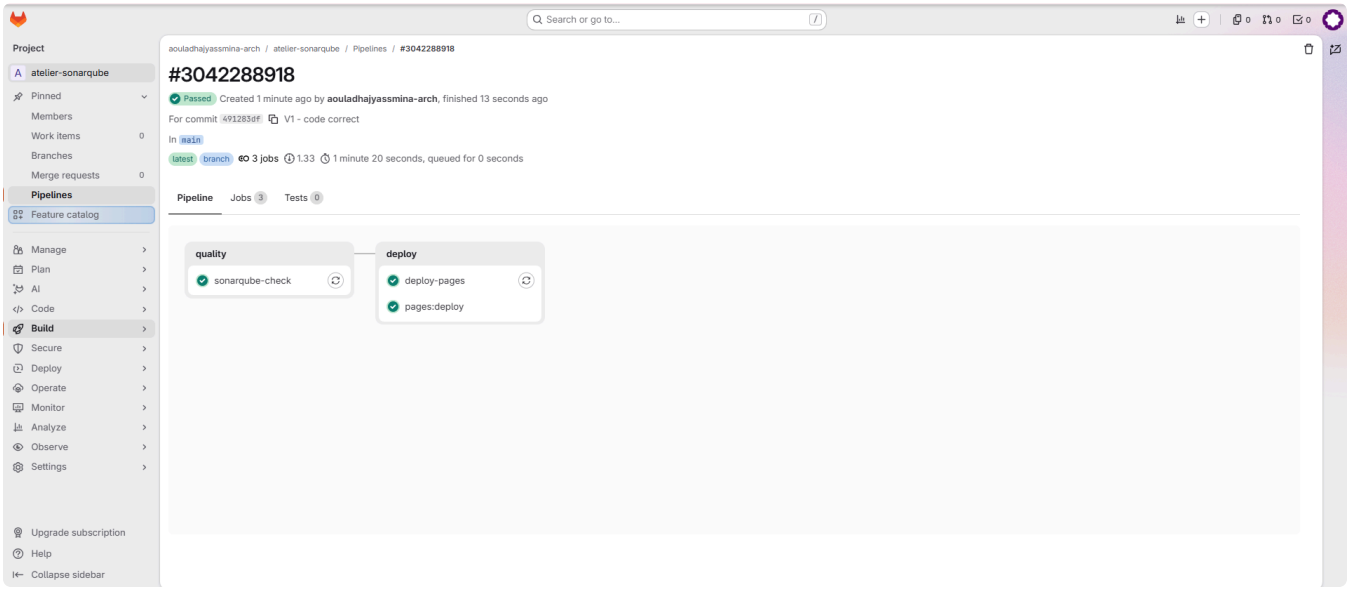
4. Réalisation

4.1 Préparation

- Création du projet public `atelier-sonarqube` sur GitLab et clonage en local.
- Création des fichiers `index.html`, `script.js` et `style.css`.
- Import du projet dans SonarQube Cloud et création du fichier `sonar-project.properties` (clé du projet, organisation, exclusions, `sonar.qualitygate.wait=true`).
- Création d'un jeton SonarQube et ajout des variables `SONAR_TOKEN` et `SONAR_HOST_URL` dans Settings → CI/CD → Variables de GitLab (masquées et protégées).
- Désactivation de l'Automatic Analysis dans SonarQube, incompatible avec l'analyse via CI.

4.2 Version 1 : code correct

La V1 affiche un message et un bouton qui modifie le texte avec `textContent`, une méthode sûre. Après le push, le pipeline passe au vert : les deux jobs réussissent, le Quality Gate est **Passed** (0 issue) et le site est publié sur GitLab Pages.



[Capture : SonarQube, Quality Gate Passed]

[Capture : site déployé sur GitLab Pages]

4.3 Version 2 : code vulnérable

Dans la V2, une saisie utilisateur est insérée dans la page avec `innerHTML`, sans nettoyage :

```
message.innerHTML = input.value;
```

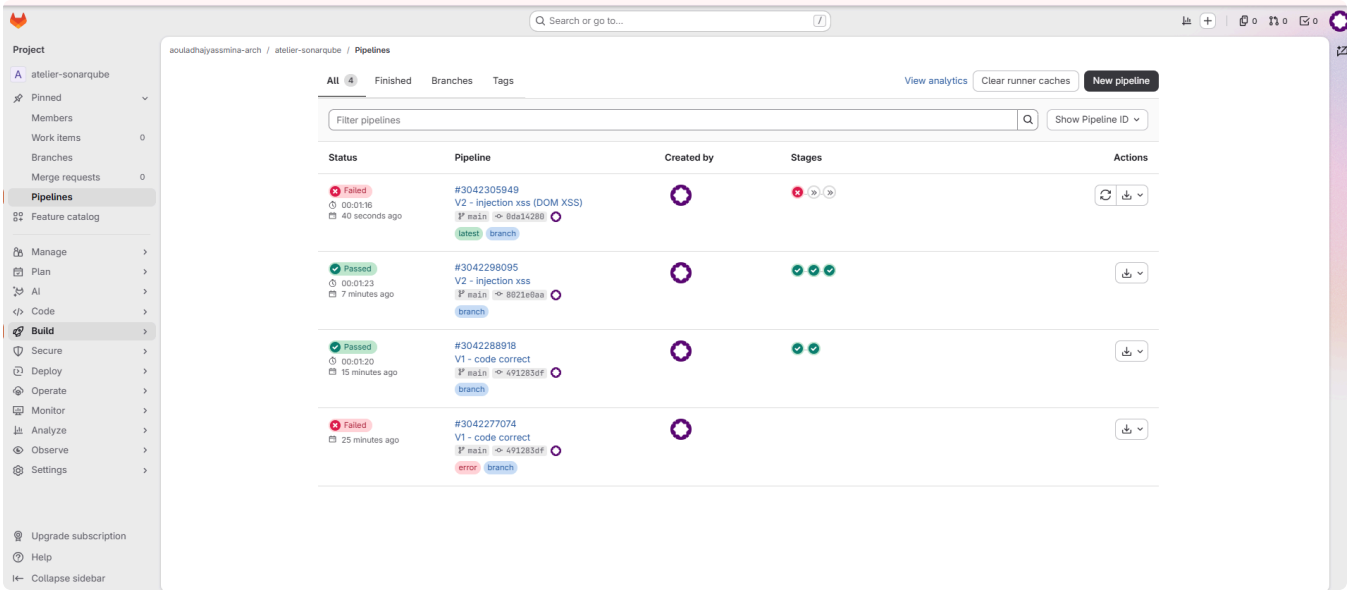
Ce code permet une injection XSS : en saisissant une balise `<img>` avec un attribut `onerror`, on peut exécuter du JavaScript arbitraire dans la page. J'ai également ajouté un second cas où une valeur lue dans l'URL (`window.location.search`) est insérée dans `innerHTML`, un schéma que SonarQube détecte systématiquement (règle de type DOM XSS).

5. Résultats

Après le push de la V2 :

- Le job `syntax-check` réussit, car le code est syntaxiquement correct.
- Le job `sonarqube-check` **échoue** : le Quality Gate est **Failed**.
- Les stages `security` et `deploy` ne s'exécutent pas : **le déploiement est bloqué**.
- Dans SonarQube, l'issue *DOM XSS via unsanitized user input* est classée **Blocker** (sécurité) dans `script.js`.

- Le site publié reste la V1 : le code vulnérable n'a jamais été mis en ligne.



Vue d'ensemble des pipelines : le premier pipeline V1 a échoué (compte à vérifier), le second est passé, puis le pipeline V2 en échec (DOM XSS) a bloqué le déploiement.

[Capture : SonarQube, Quality Gate Failed]

[Capture : issue DOM XSS détaillée]

6. Difficultés rencontrées et solutions

Problème	Solution
`git push` refusé (compte authentifié par SSO)	Utilisation d'un Personal Access Token avec le scope `write_repository` à la place du mot de passe
Erreur « Automatic Analysis is enabled »	Désactivation de l'Automatic Analysis dans Administration → Analysis method
Pipeline bloqué (« verify your account »)	Vérification du compte GitLab, puis relance du pipeline
Jeton refusé (« Token was revoked »)	Création d'un nouveau jeton, car un jeton exposé publiquement est révoqué
V2 acceptée par SonarQube avec `input.value` seul	Ajout d'un cas où la donnée provient de l'URL, détecté comme DOM XSS

7. Bonnes pratiques de sécurité retenues

- Ne jamais écrire un jeton dans le code, un message ou une capture d'écran, et le révoquer s'il a été exposé.
- Stocker les secrets dans des variables CI/CD masquées.
- Préférer `textContent` à `innerHTML` pour afficher des données non fiables.
- Utiliser un Quality Gate pour bloquer automatiquement le déploiement d'un code non conforme.

8. Compétences acquises

- Écrire un pipeline GitLab CI/CD en plusieurs stages.
- Intégrer SonarQube Cloud et interpréter le Quality Gate, les issues et les niveaux de gravité.
- Déployer un site statique sur GitLab Pages.
- Comprendre et reproduire une faille XSS, puis l'identifier avec un outil d'analyse.
- Diagnostiquer des erreurs d'authentification et de configuration.

9. Conclusion

Ce projet montre qu'un pipeline CI/CD couplé à SonarQube automatise les contrôles et protège la mise en production : le code conforme est déployé, le code vulnérable est bloqué. Cela ne remplace ni les tests ni la revue de code, mais constitue un filet de sécurité précieux dès les premières étapes du développement.

Pour corriger la V2, il suffit de remplacer `innerHTML` par `textContent`, puis de pousser à nouveau : le Quality Gate repasse au vert et le déploiement reprend.